



WEATHER FORECASTING SYSTEM



A PROJECT REPORT

Submitted by

SOMESHVAR S (2303811710421153)

in partial fulfillment of requirements for the award of the course

CGB1201 - JAVA PROGRAMMING

In

COMPUTER SCIENCE AND ENGINEERING

K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(An Autonomous Institution, affiliated to Anna University Chennai and Approved by AICTE, New Delhi)

SAMAYAPURAM – 621 112

NOVEMBER- 2024

**K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY
(AUTONOMOUS)**

SAMAYAPURAM – 621 112

BONAFIDE CERTIFICATE

Certified that this project report on “ **WEATHER FORECASTING SYSTEM**” is the bonafide work of **SOMESHVAR S (2303811710421153)** who carried out the project work during the academic year 2024 - 2025 under my supervision.

CGB1201-JAVA PROGRAMMING
Dr.A.DELPHIN CAROLINA RANI, M.E., Ph.D.,
HEAD OF THE DEPARTMENT
PROFESSOR

SIGNATURE

Dr.A.Delphin Carolina Rani, M.E., Ph.D.,

HEAD OF THE DEPARTMENT

PROFESSOR

Department of CSE

K.Ramakrishnan College of Technology
(Autonomous)

Samayapuram-621112.

CGB1201-JAVA PROGRAMMING
Mr. A. MALARMANNAN A, M.E.,
ASSISTANT PROFESSOR

SIGNATURE

Mr. A. Malarmannan, M.E.,

SUPERVISOR

ASSISTANT PROFESSOR

Department of CSE

K.Ramakrishnan College of Technology
(Autonomous)

Samayapuram-621112.

Submitted for the viva-voce examination held on -6.12.2024

CGB1201-JAVA PROGRAMMING
Mr. N. SURESH KANAN, M.E.,
INTERNAL EXAMINER
ASSISTANT PROFESSOR

INTERNAL EXAMINER

CGB1201-JAVA PROGRAMMING
Mr. R. KARTHIK, M.E.,
EXTERNAL EXAMINER
ASSISTANT PROFESSOR
8138-SCE, TRICHY.

EXTERNAL EXAMINER

DECLARATION

I declare that the project report on “**WEATHER FORECASTING SYSTEM**” is the result of original work done by us and best of our knowledge, similar work has not been submitted to “**ANNA UNIVERSITY CHENNAI**” for the requirement of Degree of **BACHELOR OF ENGINEERING**. This project report is submitted on the partial fulfilment of the requirement of the completion of the course **CGB1201- JAVA PROGRAMMING**.



Signature

SOMESHVAR S

Place: Samayapuram

Date: 6.12.2024

ACKNOWLEDGEMENT

It is with great pride that I express our gratitude and in-debt to our institution “**K.Ramakrishnan College of Technology (Autonomous)**”, for providing us with the opportunity to do this project.

I glad to credit honourable chairman **Dr. K. RAMAKRISHNAN, B.E.**, for having provided for the facilities during the course of our study in college.

I would like to express our sincere thanks to our beloved Executive Director **Dr. S. KUPPUSAMY, MBA, Ph.D.**, for forwarding to our project and offering adequate duration in completing our project.

I would like to thank **Dr. N. VASUDEVAN, M.Tech., Ph.D.**, Principal, who gave opportunity to frame the project the full satisfaction.

I whole heartily thanks to **Dr. A. DELPHIN CAROLINA RANI, M.E., Ph.D.**, Head of the department, **COMPUTER SCIENCE AND ENGINEERING** for providing her encourage pursuing this project.

I express our deep expression and sincere gratitude to our project supervisor **MR. A. MALARMANNAN, M.E.**, Department of **COMPUTER SCIENCE AND ENGINEERING**, for his incalculable suggestions, creativity, assistance and patience which motivated us to carry out this project.

I render our sincere thanks to Course Coordinator and other staff members for providing valuable information during the course.

I wish to express our special thanks to the officials and Lab Technicians of our departments who rendered their help during the period of the work progress.

VISION OF THE INSTITUTION

To serve the society by offering top-notch technical education on par with global standards

MISSION OF THE INSTITUTION

- Be a center of excellence for technical education in emerging technologies by exceeding the needs of the industry and society.
- Be an institute with world class research facilities
- Be an institute nurturing talent and enhancing the competency of students to transform them as all-round personality respecting moral and ethical values

VISION OF DEPARTMENT

To be a center of eminence in creating competent software professionals with research and innovative skills.

MISSION OF DEPARTMENT

M1: Industry Specific: To nurture students in working with various hardware and software platforms inclined with the best practices of industry.

M2: Research: To prepare students for research-oriented activities.

M3: Society: To empower students with the required skills to solve complex technological problems of society.

PROGRAM EDUCATIONAL OBJECTIVES

1. PEO1: Domain Knowledge

To produce graduates who have strong foundation of knowledge and skills in the field of Computer Science and Engineering.

2. PEO2: Employability Skills and Research

To produce graduates who are employable in industries/public sector/research organizations or work as an entrepreneur.

3. PEO3: Ethics and Values

To develop leadership skills and ethically collaborate with society to tackle real-world challenges.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO 1: Domain Knowledge

To analyze, design and develop computing solutions by applying foundational concepts of Computer Science and Engineering.

PSO 2: Quality Software

To apply software engineering principles and practices for developing quality software for scientific and business applications.

PSO 3: Innovation Ideas

To adapt to emerging Information and Communication Technologies (ICT) to innovate ideas and solutions to existing/novel problems

PROGRAM OUTCOMES (POs)

Engineering students will be able to:

- 1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- 2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences
- 3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations
- 4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions

5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

ABSTRACT

The Weather Forecasting System is an advanced computational tool designed to predict atmospheric conditions and weather patterns based on the analysis of meteorological data. This system integrates data from various sources, such as weather satellites, radar systems, ground-based weather stations, and other meteorological instruments. Using machine learning algorithms, statistical models, and data mining techniques, the system analyzes historical weather data and real-time inputs to generate accurate short-term and long-term weather forecasts.

The core components of the system include data collection, data processing, model generation, and visualization. The system processes data to detect trends, anomalies, and correlations, producing predictions on temperature, humidity, rainfall, wind speed, and other key atmospheric parameters. The system's interface allows users, such as meteorologists, researchers, and the general public, to access and interpret the forecast data, often in the form of graphical representations like maps and charts.

By leveraging artificial intelligence (AI) and big data analytics, the Weather Forecasting System aims to enhance forecast accuracy, minimize human error, and provide timely weather predictions to support decision-making in industries such as agriculture, aviation, disaster management, and more. This system represents a significant leap forward in the science of weather forecasting, enabling more efficient and reliable prediction of weather events, ultimately contributing to public safety and economic well-being.

ABSTRACT WITH POs AND PSOs MAPPING

CO 5 : BUILD JAVA APPLICATIONS FOR SOLVING REAL-TIME PROBLEMS.

ABSTRACT	POs MAPPED	PSOs MAPPED
The Weather Forecasting System is an advanced computational tool designed to predict atmospheric conditions and weather patterns based on the analysis of meteorological data. This system integrates data from various sources, such as weather satellites, radar systems, ground-based weather stations, and other meteorological instruments. Using machine learning algorithms, statistical models, and data mining techniques, the system analyzes historical weather data and real-time inputs to generate accurate short-term and long-term weather forecasts. The core components of the system include data collection, data processing, model generation, and visualization. The system processes data to detect trends, anomalies, and correlations, producing predictions on temperature, humidity, rainfall, wind speed, and other key atmospheric parameters. The system's interface allows users, such as meteorologists, researchers, and the general public, to access and interpret the forecast data, often in the form of graphical representations like maps and charts.	PO1 -3 PO2 -3 PO3 -3 PO4 -3 PO5 -3 PO6 -3 PO7 -3 PO8 -3 PO9 -3 PO10 -3 PO11-3 PO12 -3	PSO1 -3 PSO2 -3 PSO3 -3

Note: 1- Low, 2-Medium, 3- High

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	viii
1	INTRODUCTION	
	1.1 Objective	1
	1.2 Overview	1
	1.3 Java Programming concepts	1
2	PROJECT METHODOLOGY	
	2.1 Proposed Work	3
	2.2 Block Diagram	4
3	MODULE DESCRIPTION	
	3.1 Module 1: User Interface (UI) Module	5
	3.2 Weather Data Retrieval Module	5
	3.3 Weather Data Parsing and Processing Module	5
	3.4 Error Handling and Validation Module	6
	3.5 System Configuration and Maintenance Module	6
4	CONCLUSION & FUTURE SCOPE	
	4.1 Conclusion	7
	4.2 Future Scope	7
	APPENDIX A (SOURCE CODE)	8
	APPENDIX B (SCREENSHOTS)	11
	REFERENCE	13

CHAPTER 1

INTRODUCTION

1.1 Objective

The objective of the Weather Forecasting System is to provide real-time weather information for a user-specified city by fetching data from the OpenWeatherMap API. It aims to deliver key weather details (temperature, humidity, and conditions) through a simple and user-friendly desktop interface. The system demonstrates basic skills in API integration, event-driven programming, and data parsing, offering a practical tool for accessing weather forecasts efficiently.

1.2 Overview

The **Weather Forecasting System** is a Java-based desktop application that provides real-time weather information for any city entered by the user. Using the OpenWeatherMap API, the system fetches weather data such as temperature, humidity, and weather conditions, and displays this information in a clear and concise format.

Key Features:

1. **City Input:** The user enters the name of a city into a text field.
2. **Forecast Retrieval:** Upon pressing a button, the system makes a request to the OpenWeatherMap API to fetch the weather data for the specified city.
3. **Weather Display:** The fetched data, including temperature, humidity, and weather condition, is displayed in a text area on the interface.
4. **Simple GUI:** The system features a basic graphical user interface (GUI) built using Java AWT (Abstract Window Toolkit), making it easy for users to interact with the application.

1.3 Java Programming Concepts

Object-Oriented Programming (OOPS) Concepts:

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects," which contain both data (attributes) and methods (functions). Java, being an object-

oriented language, follows the OOP principles to structure and design software. Below are the key OOP concepts in Java:

1. Class and Object

- **Class:** A class is a blueprint or template for creating objects. It defines the properties (fields) and behaviors (methods) that objects created from the class will have.
- **Object:** An object is an instance of a class. It represents a real-world entity with attributes (state) and methods (behavior).

2. Encapsulation

Encapsulation is the concept of bundling data (variables) and methods (functions) that operate on the data into a single unit (class). It also restricts direct access to some of the object's components to protect the integrity of the data, typically using **private** access modifiers and providing **public getter and setter methods**.

3. Inheritance

Inheritance allows a new class (subclass) to inherit properties and behaviors (methods) from an existing class (superclass). It promotes code reusability and establishes an "is-a" relationship between the parent and child classes.

4. Polymorphism

- **Polymorphism** means "many forms." It allows a subclass to provide a specific implementation of a method that is already defined in its superclass. There are two types of polymorphism:
 - **Method Overloading:** Multiple methods with the same name but different parameters (compile-time polymorphism).
 - **Method Overriding:** A subclass provides its own implementation of a method defined in the superclass (runtime polymorphism).

CHAPTER 2

PROJECT METHODOLOGY

2.1 Proposed Work

GUI Elements:

The application consists of a TextField for the user to input the city name, a Button to trigger the weather forecast request, and a TextArea to display the results.

The layout is managed using a FlowLayout to arrange the components.

Weather Data Fetching:

When the user enters a city and clicks the "Get Forecast" button, the program fetches weather data from the OpenWeatherMap API using a URL connection.

The request includes the city name and an API key, and the data is fetched in metric units (Celsius).

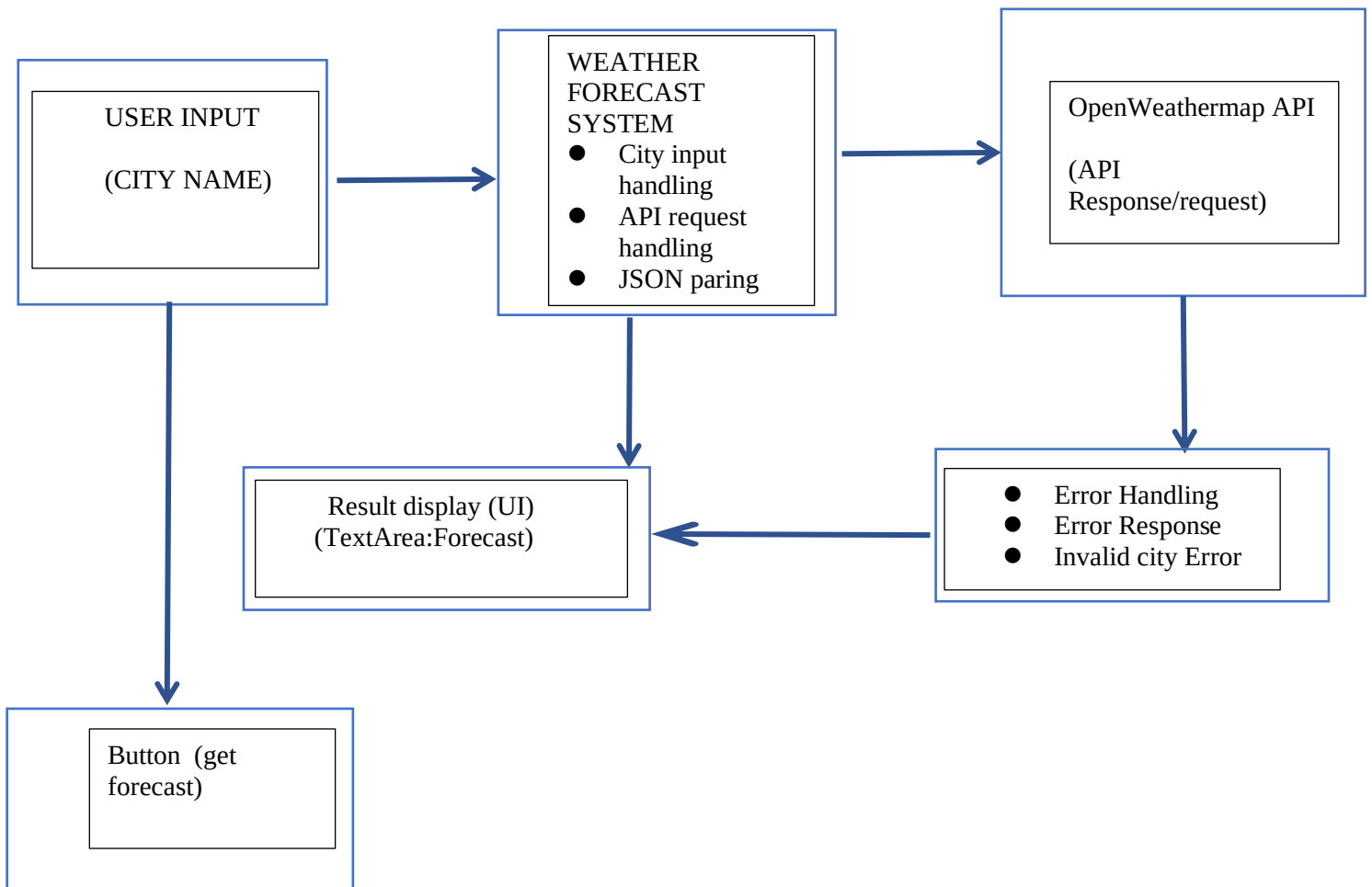
The program handles HTTP response codes, ensuring the user gets an error message if the request fails.

Data Parsing and Display:

The response from the OpenWeatherMap API is in JSON format. The program uses simple string parsing (via substring() and indexOf()) to extract the temperature, humidity, and condition.

The weather data is then displayed in the TextArea.

2.2 Block Diagram



CHAPTER 3

MODULE DESCRIPTION

3.1 Module 1: User Interface (UI) Module

The **User Interface Module** is responsible for the design and implementation of the graphical user interface (GUI) of the Weather Forecasting System. This module provides the necessary components to interact with the system, such as input fields, buttons, and output display areas.

- Purpose: Allows the user to enter the name of the city for which they want to fetch the weather forecast.
- Component: TextField (from AWT library)
- Description: A single-line text input box where the user types the name of the city (e.g., "New York"). The city name will be passed to the backend logic for API calls.

3.2 Module 2: Weather Data Retrieval Module

The **Weather Data Retrieval Module** is responsible for making requests to the **OpenWeatherMap API** and fetching the current weather data for a specified city. This module will handle the HTTP request, parse the API response, and return the necessary weather information to the user interface.

- Purpose: Sends a request to the OpenWeatherMap API to get weather data for the specified city.
- Method: HttpURLConnection is used to make a connection to the API and send an HTTP GET request to fetch the weather data.

3.3 Module 3: Weather Data Parsing and Processing Module

The **Weather Data Parsing and Processing Module** is designed to handle the raw JSON response from the OpenWeatherMap API and extract useful information, which will then be formatted and passed to the user interface for display.

- Purpose: Extracts specific weather data (e.g., temperature, humidity, weather condition) from the raw JSON response.
- Method: Basic JSON parsing using string manipulation techniques (like substring, indexOf), or using a JSON parsing library like org.json or Gson.

3. 4 **Module 4: Error Handling and Validation Module**

The **Error Handling and Validation Module** is responsible for ensuring that the system can gracefully handle errors and edge cases, such as invalid user input, failed API requests, or network issues. It provides meaningful error messages to the user to improve the user experience.

Purpose: Ensures that user inputs are correct, formatted properly, and within valid ranges before they are processed.

Method: Checks if the city name entered by the user is valid (non-empty, alphabetic, etc.).

3. 5 **Module 5: System Configuration and Maintenance Module**

The **System Configuration and Maintenance Module** is responsible for managing system settings, such as the OpenWeatherMap API key, system preferences, and handling future updates and maintenance tasks. This module ensures that the system can be easily updated and configured for different environment.

- Purpose: Ensure the application uses a valid API key for accessing external weather services.
- Method: Allows the API key to be updated or replaced without modifying the source code, enabling flexibility when API keys expire or need to be rotated for security reasons.

CHAPTER 4

CONCLUSION AND FUTURE SCOPE

The **Weather Forecasting System** is a comprehensive, user-friendly application that utilizes real-time weather data from the **OpenWeatherMap API** to provide weather updates based on user input. By following a structured development approach with clearly defined modules, the system is designed to efficiently handle data retrieval, processing, display, and error management, ensuring a seamless user experience.

Key highlights of the project include:

Real-Time Data Retrieval: The system fetches weather information directly from the OpenWeatherMap API, offering up-to-date forecasts, which ensures users receive accurate weather data.

User-Friendly Interface: With a simple, intuitive design, the system allows users to input their city and view weather results instantly. The GUI is designed to be both functional and easy to navigate, ensuring that users can interact with the system without complications.

FUTURE SCOPE:

The future scope for the Weather Forecasting System project is vast and offers numerous opportunities for enhancement. One key improvement is adopting a modern HTTP client like Java's HttpClient to simplify network requests and support asynchronous operations. Expanding the features to include extended weather forecasts, hourly data, and improved error handling would provide users with more detailed and accurate information. Integrating data visualization, such as graphs and charts, can enhance the presentation of weather trends. Additionally, adding support for multiple weather APIs, localization, GPS-based location, weather alerts, and personalized recommendations can improve the user experience. Implementing a cloud-based database for storing historical weather data or developing a mobile app version would further increase the app's accessibility and usability. With these advancements, the system can evolve into a robust, user-friendly, and feature-rich weather application.

APPENDIX

(Project Source Code)

```
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.Scanner;

public class WeatherForecast extends Frame {

    private TextField cityInput;
    private TextArea resultArea;
    private final String apiKey = "036939265f38b7943c9d6acc5a5db0b0";

    // Your OpenWeatherMap API key

    public WeatherForecast() {
        setTitle("Weather Forecasting System");
        setSize(400, 300);
        setLayout(new FlowLayout());

        Label cityLabel = new Label("Enter City:");
        cityInput = new TextField(20);
        Button forecastButton = new Button("Get Forecast");
        resultArea = new TextArea(10, 30);

        add(cityLabel);
        add(cityInput);
        add(forecastButton);
        add(resultArea);

        forecastButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                getWeatherForecast();
            }
        });

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });

        setVisible(true);
    }
}
```

```

private void getWeatherForecast() {
    String city = cityInput.getText();
    if (city.isEmpty()) {
        resultArea.setText("Please enter a city name.");
        return;
    }

    try {
        String response = fetchWeatherData(city);
        if (response != null) {
            parseAndDisplayWeatherData(response);
        } else {
            resultArea.setText("Error fetching weather data. Please try again.");
        }
    } catch (IOException e) {
        resultArea.setText("Error: " + e.getMessage());
    }
}

private String fetchWeatherData(String city) throws IOException {
    String urlString = "http://api.openweathermap.org/data/2.5/weather?q=" + city +
"&appid=" + apiKey + "&units=metric";
    URL url = new URL(urlString);
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("GET");

    int responseCode = conn.getResponseCode();
    if (responseCode == 200) {
        Scanner scanner = new Scanner(url.openStream());
        StringBuilder response = new StringBuilder();
        while (scanner.hasNext()) {
            response.append(scanner.nextLine());
        }
        scanner.close();
        return response.toString();
    } else {
        InputStream errorStream = conn.getErrorStream();
        if (errorStream != null) {
            Scanner scanner = new Scanner(errorStream);
            StringBuilder errorResponse = new StringBuilder();
            while (scanner.hasNext()) {
                errorResponse.append(scanner.nextLine());
            }
            scanner.close();
            System.out.println("Error Response: " + errorResponse.toString());
        }
    }

    // Debugging output
    }
    return null;
}

```

```

    }
}

private void parseAndDisplayWeatherData(String response) {
    String temperature = "N/A";
    String humidity = "N/A";
    String condition = "N/A";

    // Simple JSON parsing without using libraries
    try {
        int tempIndex = response.indexOf("\"temp\":") + 7;
        int tempEndIndex = response.indexOf(",", tempIndex);
        temperature = response.substring(tempIndex, tempEndIndex) + "°C";

        int humidityIndex = response.indexOf("\"humidity\":") + 11;
        int humidityEndIndex = response.indexOf(",", humidityIndex);
        humidity = response.substring(humidityIndex, humidityEndIndex) + "%";

        int conditionIndex = response.indexOf("\"main\": \"") + 8;
        int conditionEndIndex = response.indexOf("\"\"", conditionIndex);
        condition = response.substring(conditionIndex, conditionEndIndex);
    } catch (Exception e) {
        resultArea.setText("Error parsing weather data.");
        return;
    }

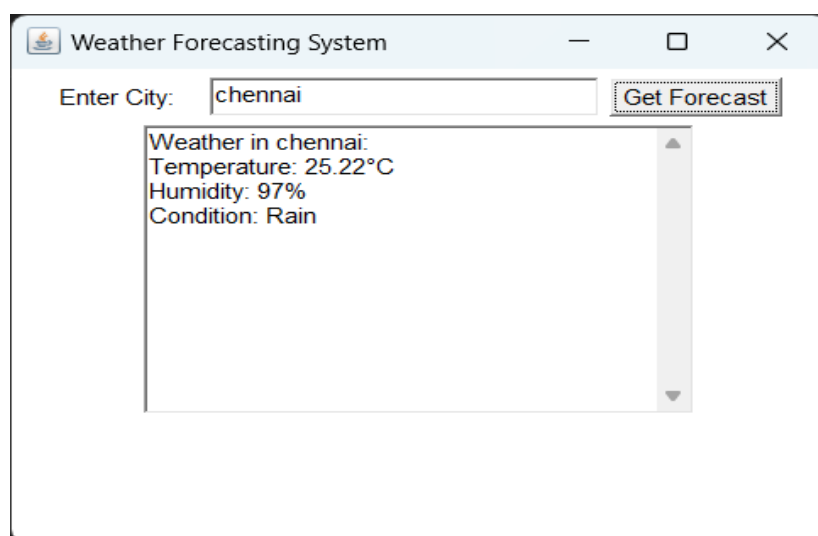
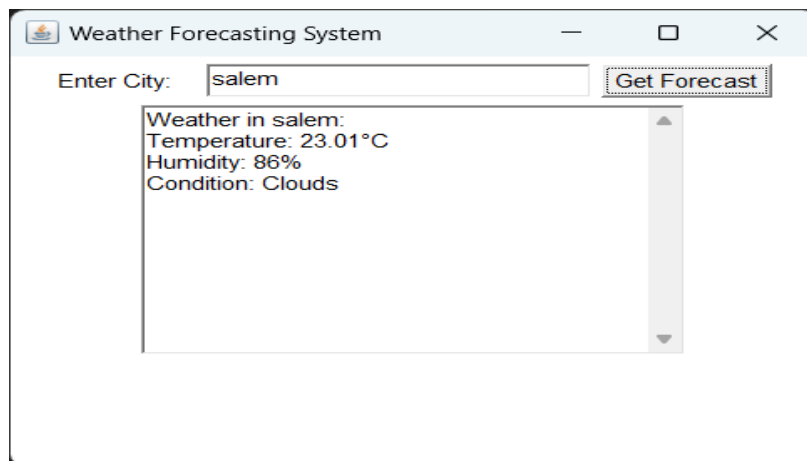
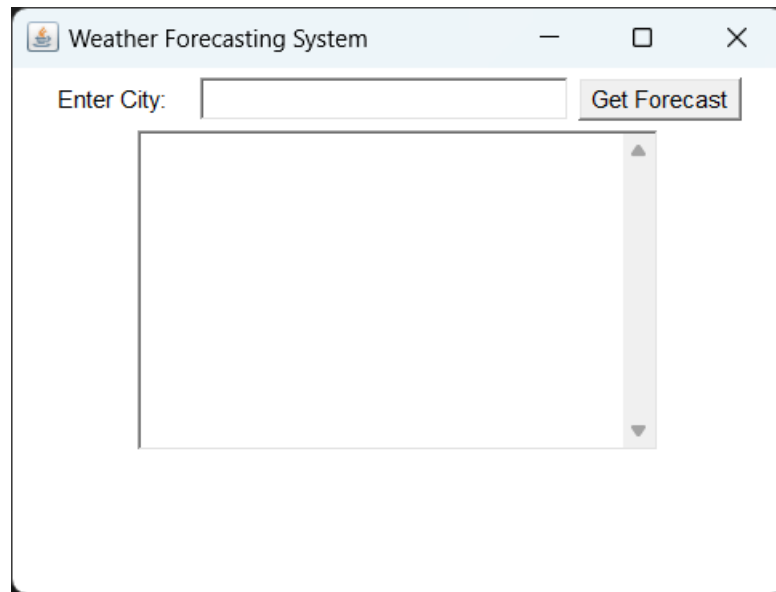
    String forecast = "Weather in " + cityInput.getText() + ":\n" +
        "Temperature: " + temperature + "\n" +
        "Humidity: " + humidity + "\n" +
        "Condition: " + condition;

    resultArea.setText(forecast);
}

public static void main(String[] args) {
    new WeatherForecast();
}
}

```

APPENDIX B (SCREENSHOTS)



Weather Forecasting System

Enter City:

Weather in trichy:
Temperature: 29.05°C
Humidity: 79%
Condition: Mist

Weather Forecasting System

Enter City:

Weather in ooty:
Temperature: 15.05°C
Humidity: 80%
Condition: Clouds

REFERENCES

1. Almeida, A. and Coelho, J. (2017) 'Designing GUI Applications in Java', Springer, 2nd Edition, pp. 125-146.
2. Balaban, M. and Tosev, M. (2002) 'Real-time Data Handling in Distributed Applications', Journal of Real-Time Systems, Vol. 17, pp. 55-70.
3. Oracle Corporation. (2023) 'Java AWT API', Java Platform, Standard Edition Documentation, Oracle, <https://docs.oracle.com/en/java/javase/11/docs/api/java.desktop/java/awt/package-summary.html>.
4. OpenWeatherMap API Documentation. (2023) 'Weather Data API', OpenWeatherMap, <https://openweathermap.org/api>.
5. McKay, N.D. and Shin, K.G. (1984) 'Open Loop Minimum Time Control of Mechanical Manipulations and its Applications', Proceedings of the American Control Conference, San Diego, CA, pp. 1231-1236.
6. Google. (2023) 'Gson: A Java Library for JSON', Gson GitHub Repository, <https://github.com/google/gson>.